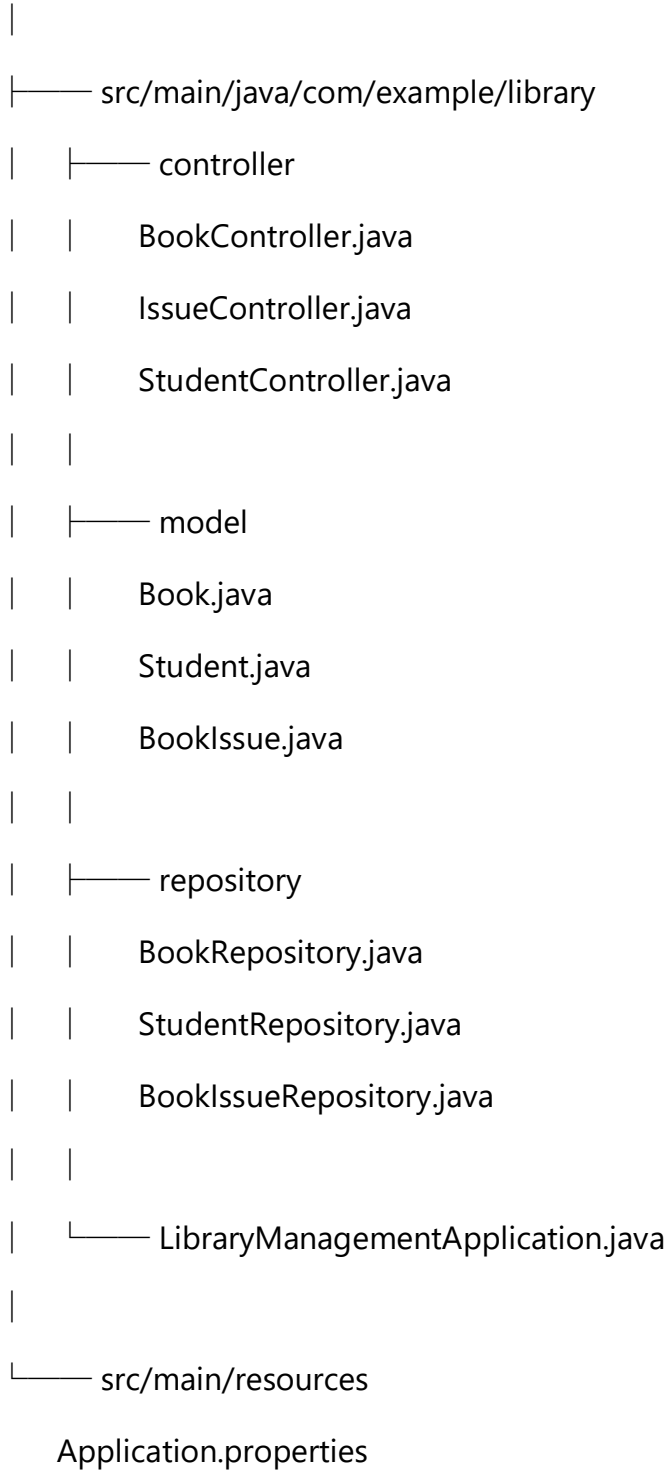


Option 1: Library Management System API

Project Structure

library-management



Step 1 Install MySQL

```
sudo apt update
```

```
sudo apt install mysql-server -y
```

Check:

```
mysql --version
```

Login:

```
sudo mysql
```

Step 2 Create Database

Inside MySQL:

```
CREATE DATABASE librarydb;
```

```
USE librarydb;
```

Task 1 Database Schema (25 Marks)

Books Table

```
CREATE TABLE books(  
    book_id INT AUTO_INCREMENT PRIMARY KEY,  
    title VARCHAR(100),  
    author VARCHAR(100),  
    category VARCHAR(50),  
    available_copies INT  
);
```

Students Table

```
CREATE TABLE students(  
    student_id INT AUTO_INCREMENT PRIMARY KEY,  
    student_name VARCHAR(100),  
    department VARCHAR(50),  
    email VARCHAR(100)  
);
```

Book Issues Table

```
CREATE TABLE book_issues(  
    issue_id INT AUTO_INCREMENT PRIMARY KEY,  
    book_id INT,  
    student_id INT,  
    issue_date DATE,  
    return_date DATE,  
  
    FOREIGN KEY(book_id)  
    REFERENCES books(book_id),  
  
    FOREIGN KEY(student_id)  
    REFERENCES students(student_id)  
);
```

Task 2 Sample Data (5 Marks)

Insert Books

```
INSERT INTO books(title,author,category,available_copies)  
VALUES  
( 'Java Programming','James Gosling','Programming',5),  
( 'Python Basics','Guido Van Rossum','Programming',4),  
( 'Database Systems','Korth','Database',3),  
( 'Operating Systems','Galvin','System',6),  
( 'Computer Networks','Tanenbaum','Networking',2);
```

Insert Students

```
INSERT INTO students(student_name,department,email)
VALUES
('Amit','MCA','amit@gmail.com'),
('Rohit','MCA','rohit@gmail.com'),
('Sneha','BCA','sneha@gmail.com'),
('Priya','BTech','priya@gmail.com'),
('Rahul','MTech','rahul@gmail.com');
```

Insert Issue Records

```
INSERT INTO book_issues(book_id,student_id,issue_date,return_date)
VALUES
(1,1,'2025-01-01','2025-01-15'),
(2,1,'2025-01-10','2025-01-20'),
(3,2,'2025-01-12','2025-01-22'),
(4,2,'2025-01-15','2025-01-25'),
(5,3,'2025-01-18','2025-01-28'),
(1,4,'2025-02-01','2025-02-15'),
(2,5,'2025-02-05','2025-02-18'),
(3,3,'2025-02-08','2025-02-20'),
(4,4,'2025-02-12','2025-02-25'),
(5,5,'2025-02-15','2025-02-28');
```

Step 3 Create Spring Boot Project

Go to workspace:

```
mkdir library-management
```

```
cd library-management
```

Generate project from Spring Initializr.

Dependencies:

Spring Web

Spring Data JPA

MySQL Driver
Lombok

application.properties

Create:

```
nano src/main/resources/application.properties
```

Paste:

```
spring.datasource.url=jdbc:mysql://localhost:3306/librarydb
spring.datasource.username=root
spring.datasource.password=
```

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

```
server.port=8080
```

Save:

CTRL+O

ENTER

CTRL+X

Create Book Entity

```
nano src/main/java/com/example/library/model/Book.java
```

```
package com.example.library.model;
```

```
import jakarta.persistence.*;
```

```
@Entity
```

```
@Table(name="books")
```

```
public class Book {
```

```
    @Id
```

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Integer bookId;

private String title;
private String author;
private String category;
private Integer availableCopies;

public Book(){}

public Integer getBookId() {
    return bookId;
}

public void setBookId(Integer bookId) {
    this.bookId = bookId;
}

public String getTitle() {
    return title;
}

public void setTitle(String title) {
    this.title = title;
}

public String getAuthor() {
    return author;
}

public void setAuthor(String author) {
    this.author = author;
}

public String getCategory() {
    return category;
}
```

```

    public void setCategory(String category) {
        this.category = category;
    }

    public Integer getAvailableCopies() {
        return availableCopies;
    }

    public void setAvailableCopies(Integer availableCopies) {
        this.availableCopies = availableCopies;
    }
}

```

Create Student Entity

```
nano src/main/java/com/example/library/model/Student.java
```

```
package com.example.library.model;
```

```
import jakarta.persistence.*;
```

```
@Entity
```

```
@Table(name="students")
```

```
public class Student {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Integer studentId;
```

```
    private String studentName;
```

```
    private String department;
```

```
    private String email;
```

```
    public Student(){}

```

```
    public Integer getStudentId() {
```

```
        return studentId;
```

```
    }
```

```
public void setStudentId(Integer studentId) {
    this.studentId = studentId;
}

public String getStudentName() {
    return studentName;
}

public void setStudentName(String studentName) {
    this.studentName = studentName;
}

public String getDepartment() {
    return department;
}

public void setDepartment(String department) {
    this.department = department;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}
}
```

Create BookIssue Entity

```
nano src/main/java/com/example/library/model/BookIssue.java
```

```
package com.example.library.model;
```

```
import jakarta.persistence.*;
import java.time.LocalDate;
```

```
@Entity
@Table(name="book_issues")
public class BookIssue {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer issueId;

    @ManyToOne
    @JoinColumn(name="book_id")
    private Book book;

    @ManyToOne
    @JoinColumn(name="student_id")
    private Student student;

    private LocalDate issueDate;
    private LocalDate returnDate;

    public BookIssue(){}

    public Integer getIssueId() {
        return issueId;
    }

    public void setIssueId(Integer issueId) {
        this.issueId = issueId;
    }

    public Book getBook() {
        return book;
    }

    public void setBook(Book book) {
        this.book = book;
    }
}
```

```

    public Student getStudent() {
        return student;
    }

    public void setStudent(Student student) {
        this.student = student;
    }

    public LocalDate getIssueDate() {
        return issueDate;
    }

    public void setIssueDate(LocalDate issueDate) {
        this.issueDate = issueDate;
    }

    public LocalDate getReturnDate() {
        return returnDate;
    }

    public void setReturnDate(LocalDate returnDate) {
        this.returnDate = returnDate;
    }
}

```

Repository Files

BookRepository

```

package com.example.library.repository;

import com.example.library.model.Book;
import org.springframework.data.jpa.repository.JpaRepository;

public interface BookRepository extends JpaRepository<Book,Integer> {
}

```

StudentRepository

```
package com.example.library.repository;

import com.example.library.model.Student;
import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository extends
JpaRepository<Student,Integer> {
}
```

BookIssueRepository

```
package com.example.library.repository;

import com.example.library.model.BookIssue;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface BookIssueRepository extends
JpaRepository<BookIssue,Integer> {

    List<BookIssue> findByStudentStudentId(Integer studentId);

}
```

API 1 Add Book

```
@PostMapping("/books")
public Book addBook(@RequestBody Book book){
    return bookRepository.save(book);
}
```

API 2 Get All Books

```
@GetMapping("/books")
public List<Book> getAllBooks(){
    return bookRepository.findAll();
}
```

API 3 Issue Book

```
@PostMapping("/issues")
public BookIssue issueBook(@RequestBody BookIssue issue){
    return issueRepository.save(issue);
}
```

API 4 Student Issue History

```
@GetMapping("/students/{id}/issues")
public List<BookIssue> getStudentHistory(@PathVariable Integer id){
    return issueRepository.findByStudentStudentId(id);
}
```

Run Project

```
./mvnw spring-boot:run
```

or

```
mvn spring-boot:run
```

Task 4 Postman Testing

Add Book

POST

<http://localhost:8080/books>

Body:

```
{
  "title":"Spring Boot",
  "author":"Craig Walls",
  "category":"Programming",
  "availableCopies":5
}
```

Expected Response:

```
{
  "bookId":6,
  "title":"Spring Boot",
  "author":"Craig Walls",
  "category":"Programming",
  "availableCopies":5
}
```

Get All Books

GET

<http://localhost:8080/books>

Issue Book

POST

<http://localhost:8080/issues>

Body:

```
{
  "book":{
    "bookId":1
  },
  "student":{
    "studentId":1
  },
  "issueDate":"2025-05-01",
}
```

```
"returnDate": "2025-05-15"  
}
```

Student History

GET

<http://localhost:8080/students/1/issues>